

**Pró-reitora Acadêmica
Escola de Educação, Tecnologia e Comunicação
Curso de Bacharelado em Ciência da Computação
Programação concorrente e distribuída**

SISTEMA DE BUSCA DISTRIBUÍDO

Autores: Gabriel de Sousa Figueredo, Caio Henrique Pereira, Cleber Duarte Silva, Ytalo Santana Ribeiro

Orientador: Prof. João Robson Santos Martins

**Brasília - DF
2025**

1. Explicação Teórica

1.1. O que é Computação Distribuída

A computação distribuída é um paradigma da ciência da computação que projeta e estuda sistemas compostos por múltiplos componentes autônomos, chamados de nós, que se comunicam e coordenam suas ações através de uma rede para atingir um objetivo comum. Em contraste com um sistema centralizado, onde uma única máquina detém todo o processamento e armazenamento, um sistema distribuído divide a carga de trabalho entre seus nós. A principal característica, conforme definido por Tanenbaum e van Steen (2007), é que, para o usuário final, o sistema se apresenta como uma única entidade, mascarando a complexidade e a heterogeneidade da infraestrutura subjacente.

O projeto atual implementa este conceito ao simular um sistema de busca onde a base de dados de artigos científicos é particionada e distribuída entre dois servidores de busca (nós B e C). Um terceiro servidor (nó A) atua como um coordenador, recebendo as solicitações de um cliente, distribuindo a tarefa de busca aos nós apropriados e, por fim, agregando os resultados parciais em uma resposta agrupada.

1.2. Escalabilidade e Tolerância a Falhas

Escalabilidade é a capacidade de um sistema de lidar com uma carga de trabalho crescente, seja adicionando mais recursos aos nós existentes (escalabilidade vertical) ou adicionando mais nós ao sistema (escalabilidade horizontal). O projeto em questão, facilita o crescimento horizontal, caso a base de artigos crescesse a ponto de a busca se tornar lenta, a arquitetura permitiria adicionar facilmente novos servidores de busca (D, E, F, etc.), cada um responsável por um novo fragmento dos dados. O Servidor A precisaria apenas ser atualizado para encaminhar as buscas para este novo conjunto de nós.

Tolerância a Falhas é a propriedade que habilita um sistema a continuar operando, possivelmente de forma degradada, mesmo na ocorrência de falhas em um ou mais de seus componentes. Na implementação atual, o sistema possui uma tolerância a falhas parcial. Se o Servidor C, por exemplo, ficasse indisponível, o Servidor A ainda conseguiria contatar o Servidor B e retornar os resultados encontrados em sua metade da base de dados. O método `consultarServidor` no Servidor A foi projetado para não travar o sistema em caso de `ConnectException`, retornando uma lista vazia e permitindo que a

operação prossiga. Embora o resultado seja incompleto, o sistema como um todo não entra em colapso. O Servidor A, no entanto, representa um ponto único de falha, pois sua indisponibilidade tornaria todo o sistema inacessível.

1.3. Vantagens e Desvantagens da Arquitetura

Vantagens:

- **Performance:** Ao distribuir a busca entre dois servidores, a tarefa pode ser paralelizada, reduzindo o tempo total de busca em comparação com um único servidor processando todos os dados.
- **Escalabilidade:** Como discutido, a arquitetura permite a adição de mais nós de busca para acomodar o crescimento da base de dados, demonstrando escalabilidade horizontal.
- **Manutenção e desacoplamento:** Cada servidor de busca pode ser mantido e atualizado de forma independente, desde que seu contrato de comunicação (formato de dados e porta) seja mantido.

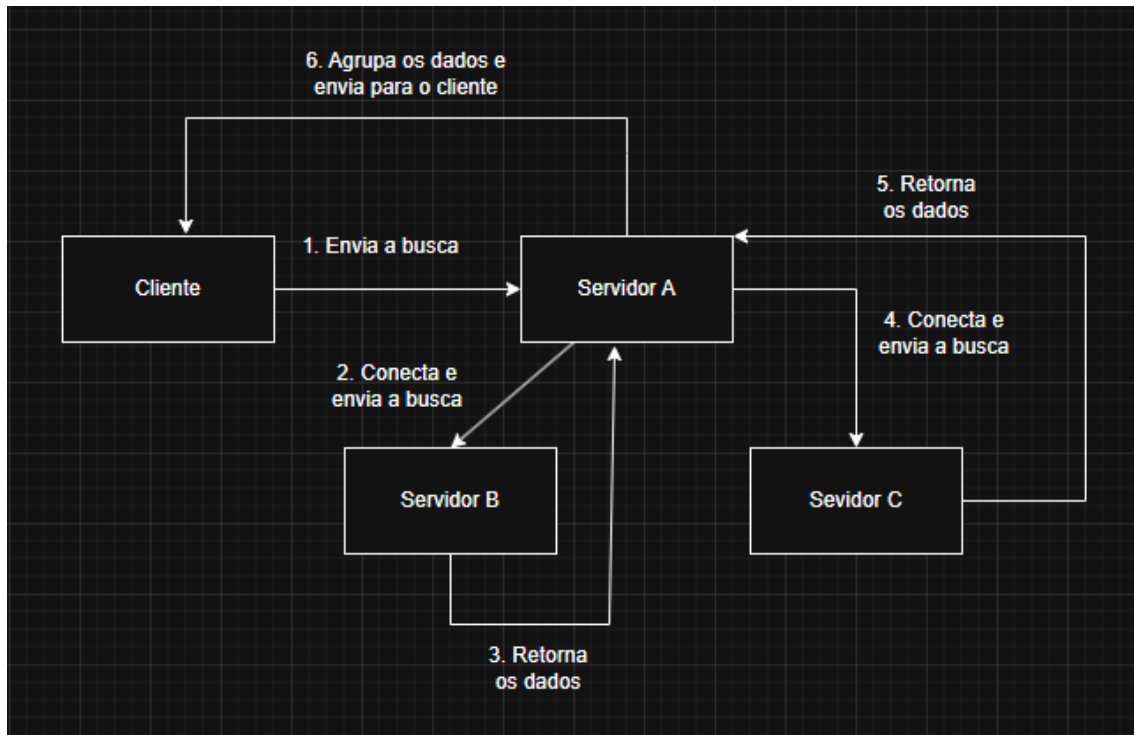
Desvantagens:

- **Complexidade de Gerenciamento:** Sistemas distribuídos são inerentemente mais complexos de implantar, monitorar e depurar do que sistemas monolíticos.
- **Latência de Rede:** A comunicação via sockets introduz um overhead de rede. O tempo gasto para o Servidor A se comunicar com B e C é adicionado ao tempo total de resposta.
- **Ponto Único de Falha:** Caso o servidor A fique inoperante, todo o sistema será afetado, pois ele é o responsável por coordenar as ações no sistema distribuído.

2. Arquitetura da Solução

2.1. Diagrama de Comunicação

O fluxo de uma solicitação de busca segue os seguintes passos:



2.2. Formato do Dado Trafegado

A comunicação entre os componentes é padronizada utilizando o formato JSON:

- **Requisição (Cliente -> Servidor A -> B/C):** Um objeto JSON simples contendo a string da busca.
 - Exemplo: `{"query": "weak gravity conjecture"}`
- **Resposta (Servidores B/C -> Servidor A -> Cliente):** Um array JSON, onde cada elemento do array é um objeto JSON que representa um artigo encontrado.
 - Exemplo: `[{"title": "Título do Artigo 1", "introduction": "Introdução..."}, {"title": "Título do Artigo 2", "introduction": "Introdução..."}]`

2.3. Explicação e Justificativa do Algoritmo de Busca

Para a tarefa de localizar uma substring nos títulos e introduções dos artigos, foi implementado e utilizado o **algoritmo de Boyer-Moore**.

O algoritmo Boyer-Moore é uma forma inteligente e muito mais rápida de encontrar palavras ou padrões dentro de um texto. Diferente de métodos mais simples, que comparam letra por letra, ele economiza tempo por não fazer verificações óbvias. Seu principal diferencial está na "regra do caractere ruim". O algoritmo compara o padrão com um trecho do texto de trás para a frente e, se um caractere não bate, ele usa uma "cola" (uma tabela pré-calculada) para saber o quanto para frente pode pular no texto. Se o caractere do texto onde a verificação falhou nem existe no padrão, o algoritmo pode saltar o padrão inteiro de uma vez.

A escolha por esse método se justifica porque, na prática, ele é extremamente rápido. Enquanto uma busca comum fica muito mais lenta conforme o texto e a palavra buscada aumentam, o Boyer-Moore, com seus "saltos inteligentes", pode na verdade dividir o esforço. Como os textos a serem pesquisados são longos (como introduções de artigos científicos), essa capacidade de pular trechos inúteis reduz drasticamente o trabalho. Isso otimiza o tempo de busca em cada servidor, fazendo com que o sistema todo funcione de forma mais eficiente.

3. Referências Bibliográficas

Tanenbaum, A. S., & Van Steen, M. (2007). *Distributed Systems: Principles and Paradigms* (2nd ed.). Pearson Prentice Hall.